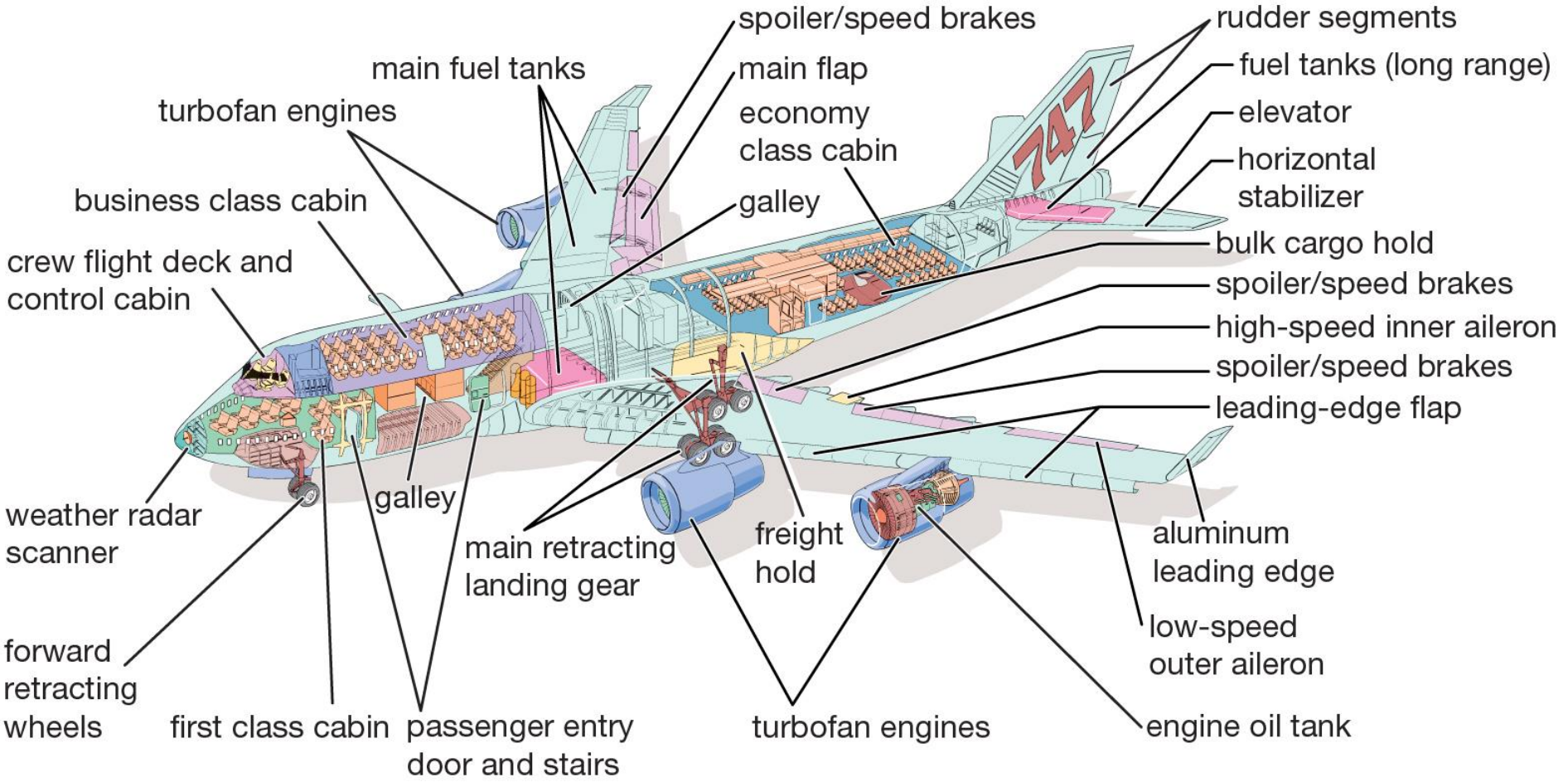
# Fundamental Principles of Object-Oriented Design

Katerina Mangaroska week 36

# Parts of a passenger jet airplane




© 2010 Encyclopædia Britannica, Inc.

# 01

# Design vs Analysis activities

Object-oriented analysis - set of analysis activities to understand needs and processes and to document those requirements in a set of specifications.

- discovery activities (facts finding activities)
- modeling and documenting system requirements

Object-oriented design – a process by which a set of detailed OO design models are built and then used by programmers to write the code, and build a system that satisfy the requirements.

- bridge between user requirements and programming the new system
- design models are often extensions of the requirements models

Design: high-level (architecture design) to low-level (components and functions of those components)

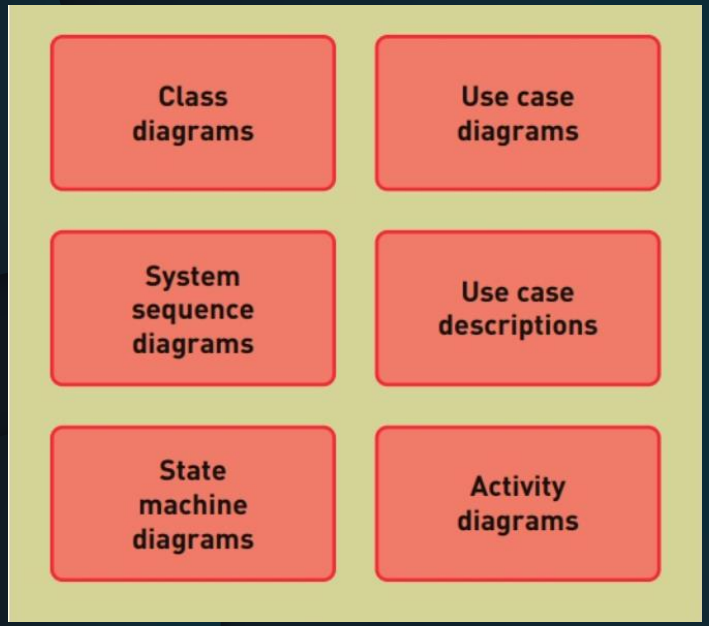
Analysis - build models to represent the real world and to understand the desired business processes and the information used in those processes.

- oriented toward business issues
- decomposition—breaking a complex problem with complicated information requirements into smaller, more understandable components.
- substantial user involvement

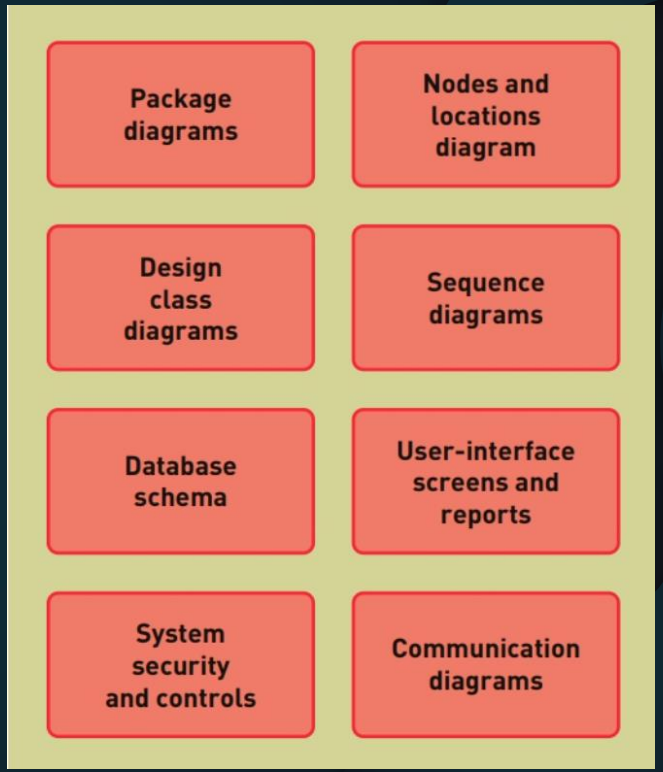
Design – convert the information gathered during analysis—the requirements models—into models that represent the solution system.

- oriented toward technical issues
- high-level (architecture) to low-level (functions of components)
- less user involvement, more involvement of systems specialists

## analysis models

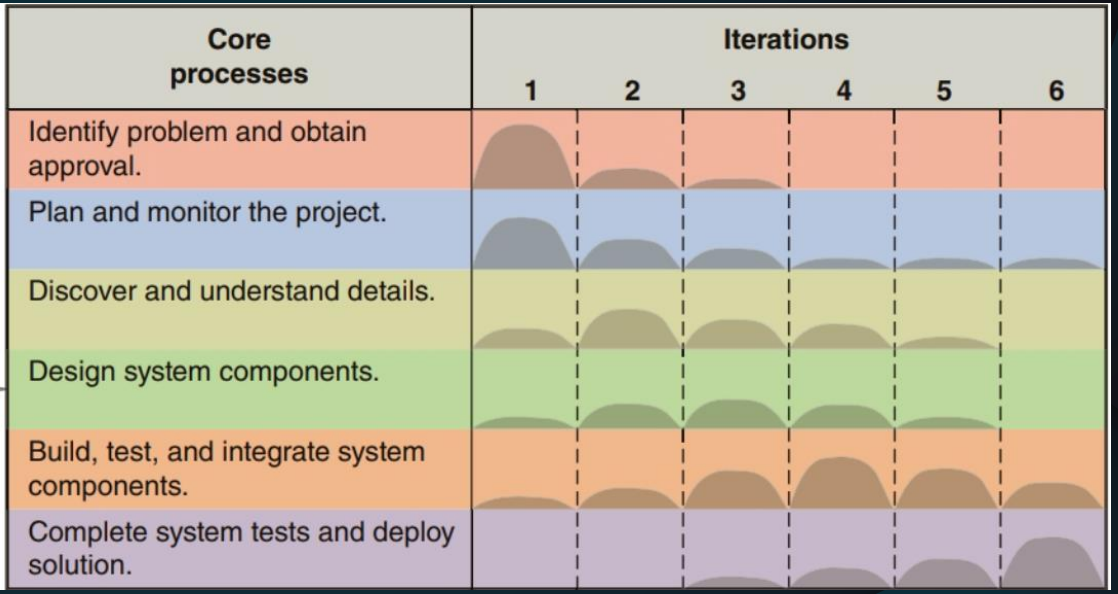



## design models



# System design activities

| Design activities |
| --- |
| Design the environment. |
| Design application architecture and software. |
| Design user interfaces. |
| Design system interfaces. |
| Design the database. |
| Design system controls and security. |



# 02
# Design activities

# 1. Design the environment

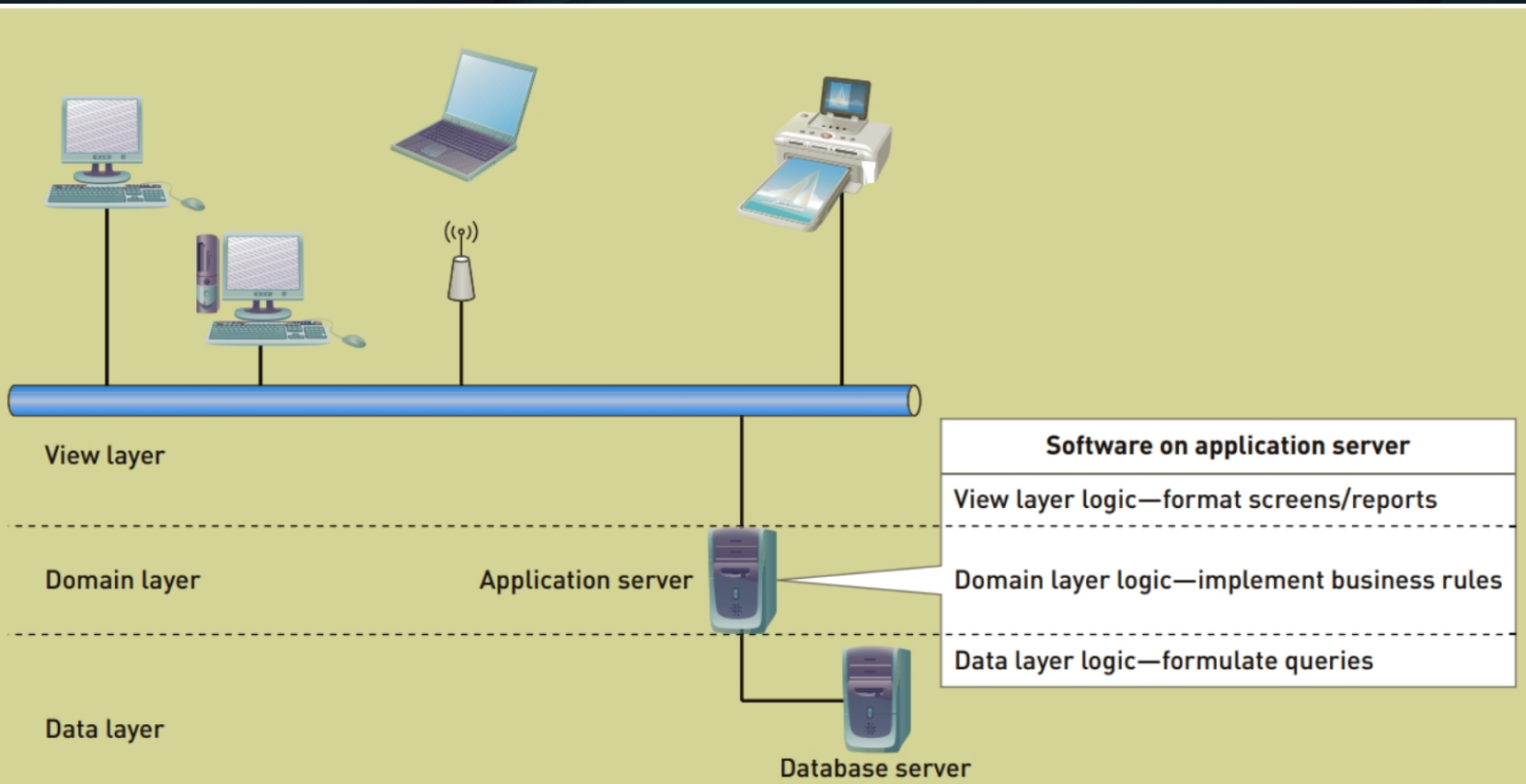
Every software application must execute in some technological environment.

- ✓ hardware - server computers, desktop computers, mobiles, firewalls, routers and cabling, fiber optics, wireless access points, etc.
- ✓ software (middleware) - operating systems, communication protocols, and other supporting software to facilitate the integration of diverse computing devices into a comprehensive solution.

e.g., web site – deployment diagram to show what HW components (nodes) exist, such as a web server, an application server, and a DB server, plus what SW components (artifacts) run on each node (e.g., web application) and how they are connected.

# 1. Design the environment

- Stand-alone applications that execute on a single computer, laptop, or mobile computing device.
- Server-based applications that utilize an application server, a database server, and content delivery network, with the users accessing the application's functions through a local browser.
- Distributed applications in which the application itself and the data execute on various computers concurrently.


View layer
Domain layer
Data layer
Application server
Database server
Software on application server
View layer logic—format screens/reports
Domain layer logic—implement business rules
Data layer logic—formulate queries

# 2. Design application architecture and software

**Application component** **is** a well-defined unit of software that performs one or more specific tasks.

- A component diagram shows the system (under development) into various high levels of functionality.
- Each component is responsible *for one clear aim* within the entire system and only interacts with other essential elements on a need-to-know basis.

# 3. Design the system interface

From the outside, components are seen as black boxes with interfaces that they use to interact with the outside world.

What is the interface between you and a TV?
- a remote control

What is the interface between two people?
- a language

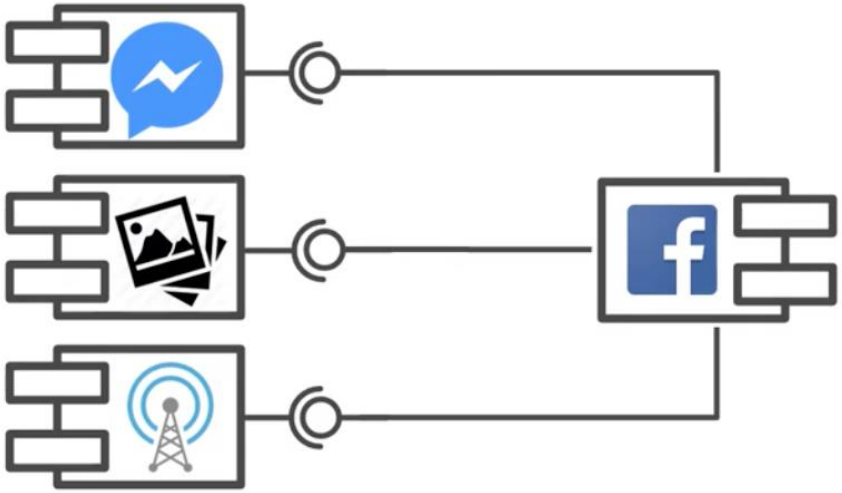
What is the interface between military people of different ranks?
- the protocol of behavior enforced in the military

Like a class, an interface defines methods. Unlike a class, an interface never implements methods; instead, classes that implement the interface implement the methods defined by the interface. A class can implement multiple interfaces.

The interface specify the dependencies at design.

# Component dependencies

- Types of dependencies
  - Processing (service/function)
  - Data (e.g. config file)
  - Infrastructure (e.g. middleware)



# 4. Design the user interface

UI design is more than just the screens. It is everything a user encounters while using the system — conceptually, perceptually, and physically.

! make or break a system ¡

What devices the users will use?

-- entering information, such as text, verbal commands, pictures, and graphics.

Computer: a GUI with windows, dialog boxes, keystrokes and mouse clicks.

Mobile phone: touch screen, screen keyboard, voice command and responses.

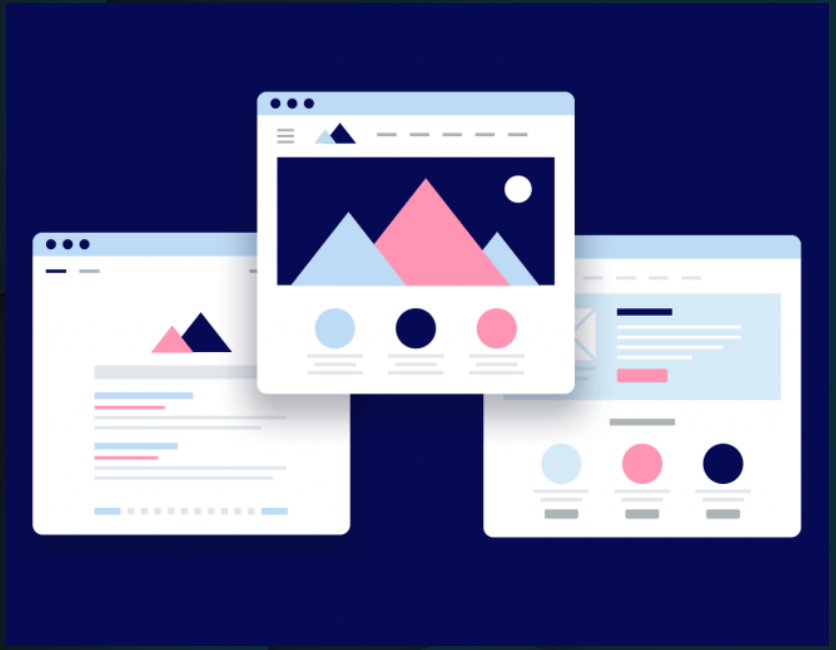
# 4. Design the user interface

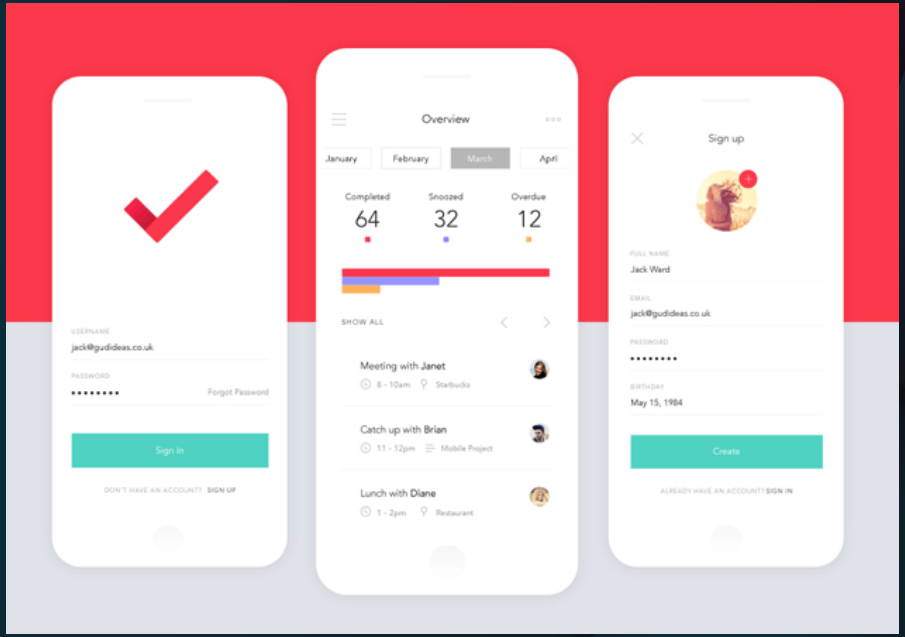
High-quality interfaces improve **employee productivity and morale**; poor-quality interfaces lead to **errors and inefficiency**.

- analysis - understand the user's needs and how the user carries out his or her job.
- design - the user interface carry the right information; it must also be ergonomically efficient and esthetically attractive.

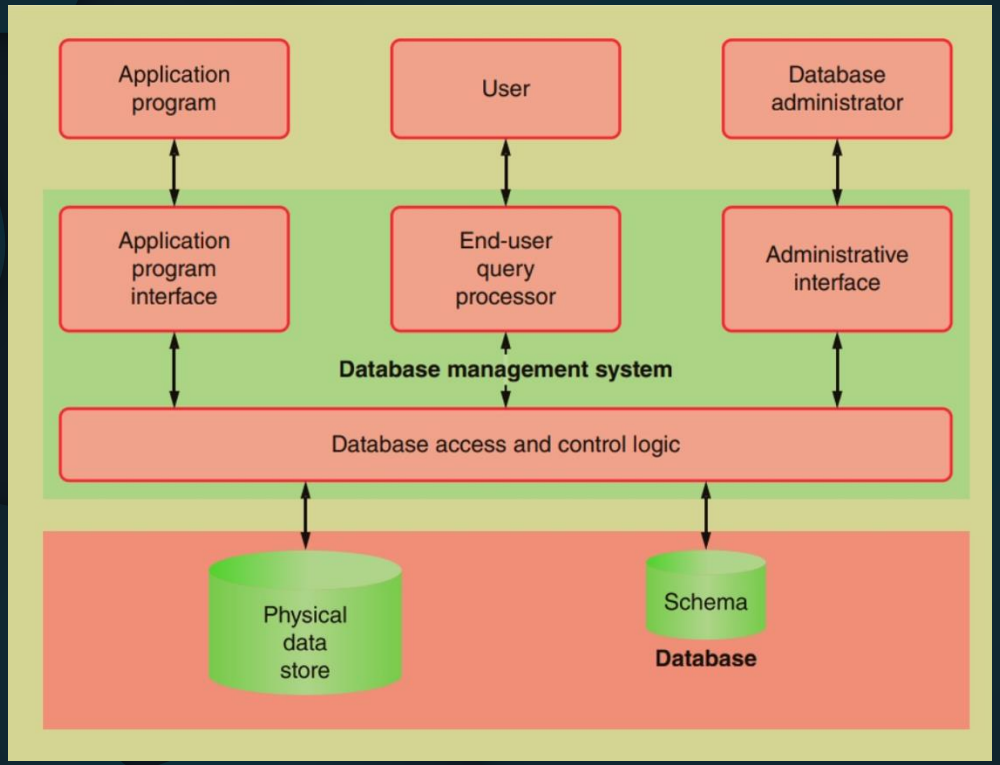
e.g., mock-ups, storyboards, graphical layouts, and prototyping with screen-modeling tools.

Users might interact with a system using one technology at their desk, another in a conference room, and yet another while traveling. Designing a single user interface is now the exception, not the norm.





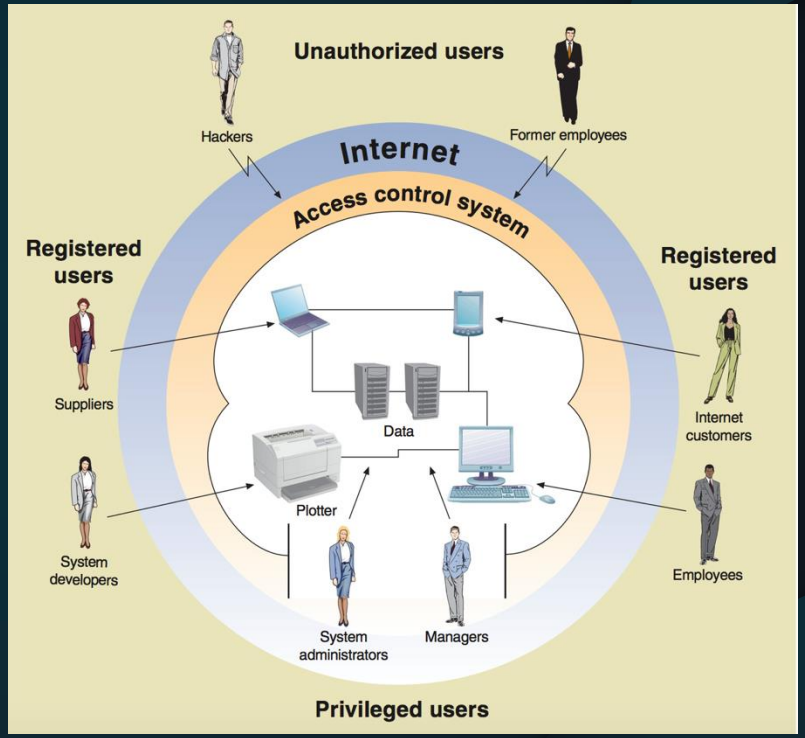
# 5. Design the database



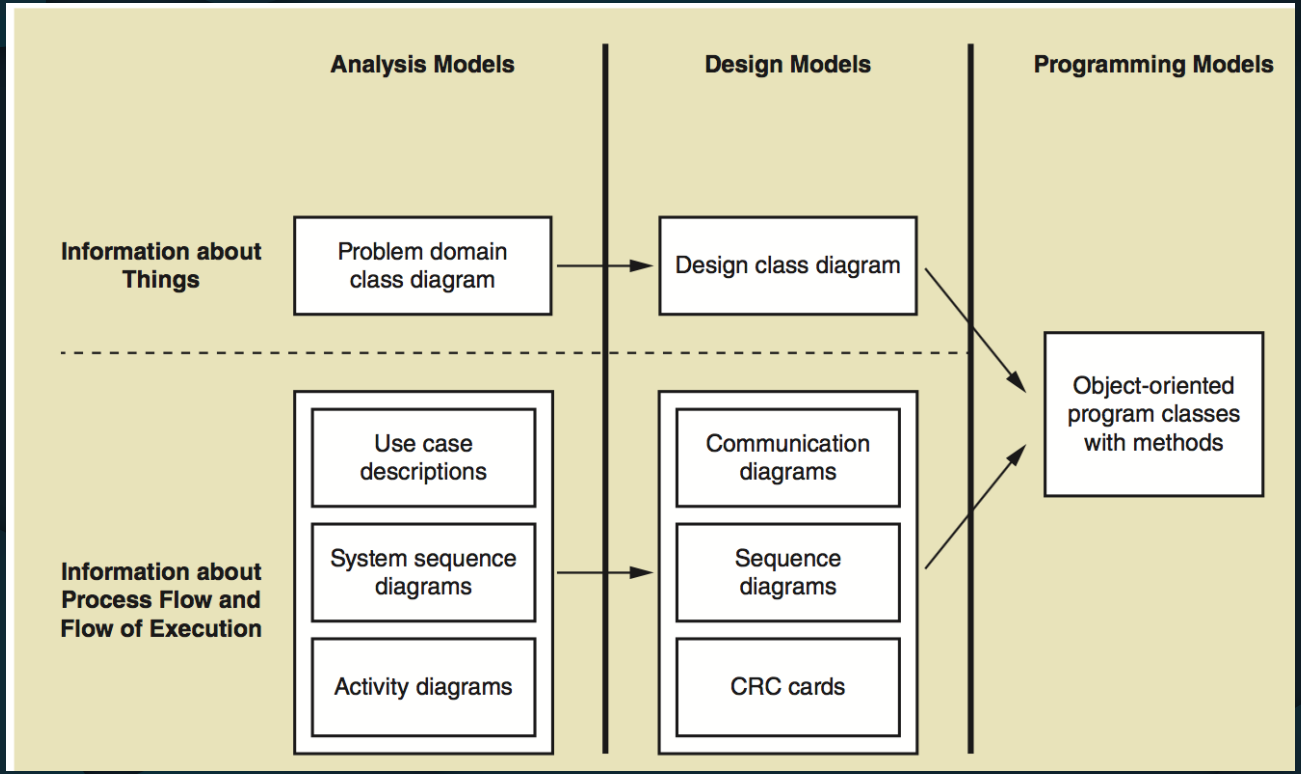
# 6. Design security and system control

Variety of risks:

- malfunction due to incorrect data input,
- inadvertently revealing confidential information,
- malicious destruction by outside parties.



# From Analysis to Design Models

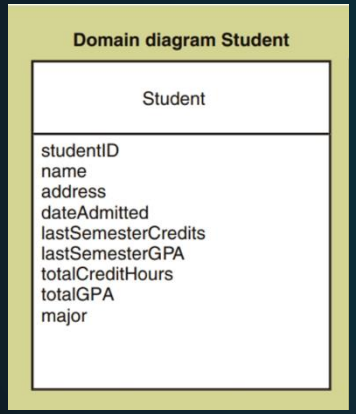


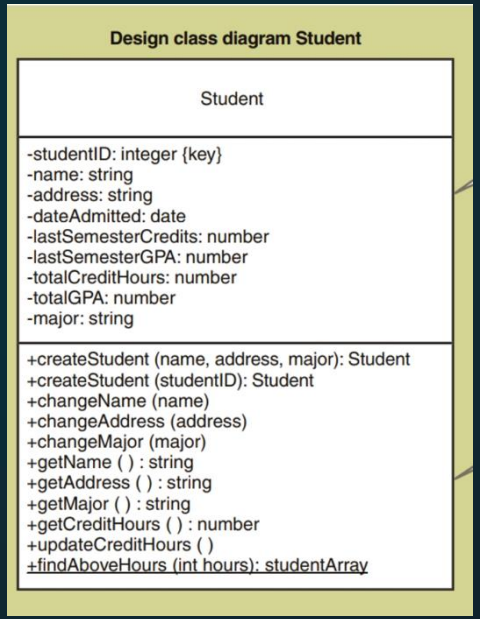
# 03

# The Design Class Diagram (Conceptual level)

# Design class diagram

It describes the set of object-oriented classes needed for programming, navigation between the classes, attribute names and properties, and method names and properties.




What concepts exist in the problem domain?

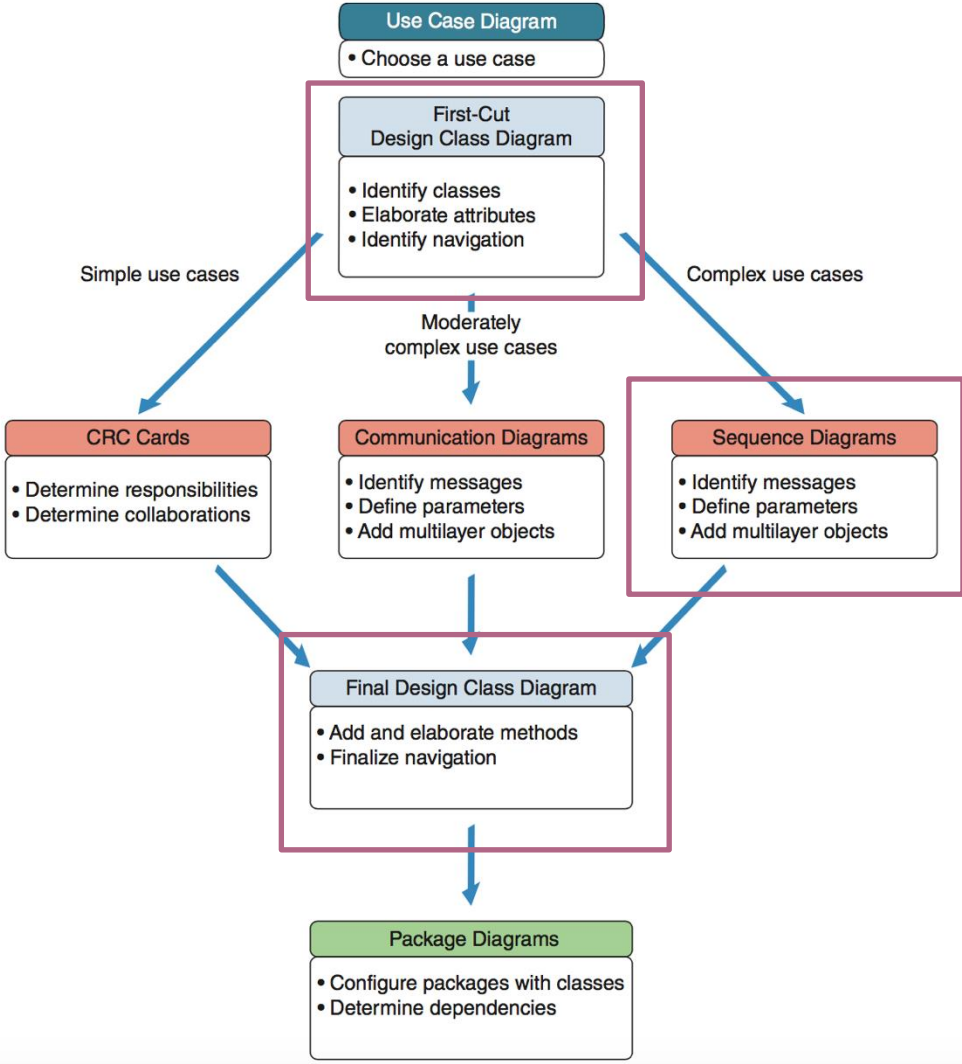
What software classes do we need, and how will they collaborate to implement the system?

*the method names given in the bottom panel of the design class diagram need to correspond to method names found in the code

# OO design process

1. Develop first-cut design class diagram (use the domain class diagram as a starting point).
2. Determine class responsibilities and class collaborations for simple use case.
3. Develop detailed sequence diagrams for complex use cases.
4. Update the design class diagram by adding methods.
5. Partition the solution into packages as appropriate.

*A package diagram can also be added to divide the classes into components or subsystems that can be implemented as a unit.

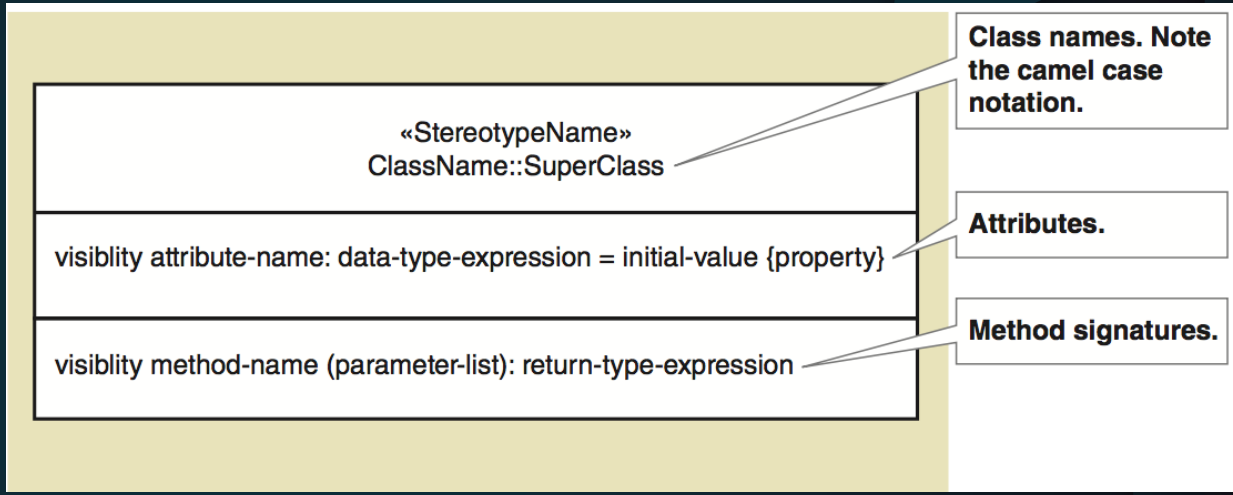


# Design class diagram

The name compartment:
stereotype name, class name, and superclass name (if any).

Attributes:
- Attribute name
- Data-type-expression (such as character, string, integer, number, currency, or date)
- Initial-value, if applicable
- Property (within curly braces), such as {key}, if applicable

Method signature:
- Method name
- Method parameter list (incoming arguments)
- Return type-expression (the type of the return parameter from the method)




**Visibility** denotes whether other objects can directly access the attribute.

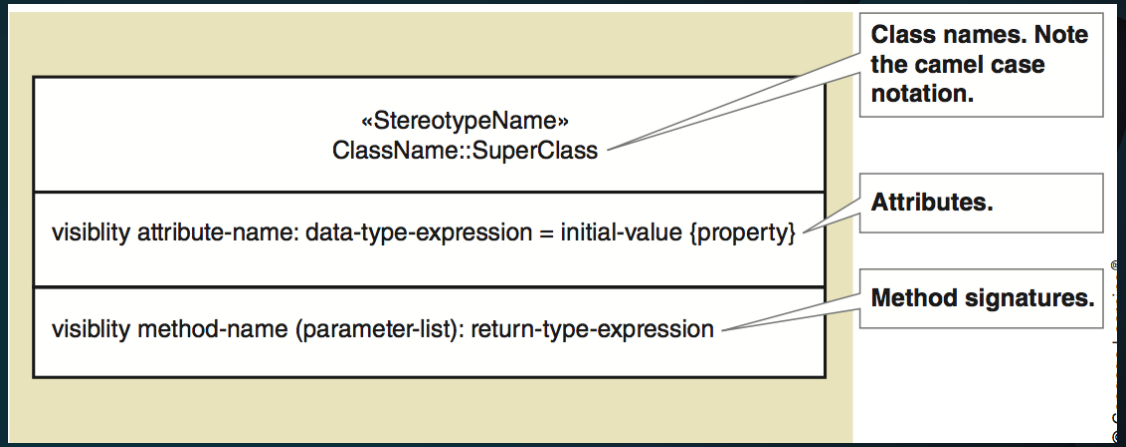
"+" is visible, "-" is invisible

# Design class diagram

The name compartment:
stereotype name, class name, and superclass name (if any).

Attributes:
- Attribute name
- Data-type-expression (such as character, string, integer, number, currency, or date)
- Initial-value, if applicable
- Property (within curly braces), such as {key}, if applicable

Method signature:
- Method name
- Method parameter list (incoming arguments)
- Return type-expression (the type of the return parameter from the method)




*[true/false condition] return-value := message-name (parameter-list)*

Message-name is the description of the requested service written as a verb-noun.
Parameter-list shows the data that are passed with the message.
Return-value describe data being returned from the destination object to the source object

# Design class stereotypes

A **stereotype** is simply a way to categorize a model element as a certain type.

1. An **entity class** is the design stereotype for a problem domain class.
   - it typically describes something users deal with when doing their work.



# Design class stereotypes

2. A **boundary** or **view class** is specifically designed to live on the system's automation boundary.

   - In a desktop system, these classes would be the **windows classes,** or all the other classes associated with the **user interface**. A boundary class could also be a system interface between a company's system and an external system.



# Design class stereotypes

3. A **controller class** mediates between the boundary classes and the entity classes.

- its responsibility is to catch the messages from the boundary class objects and send them to the correct entity class objects.



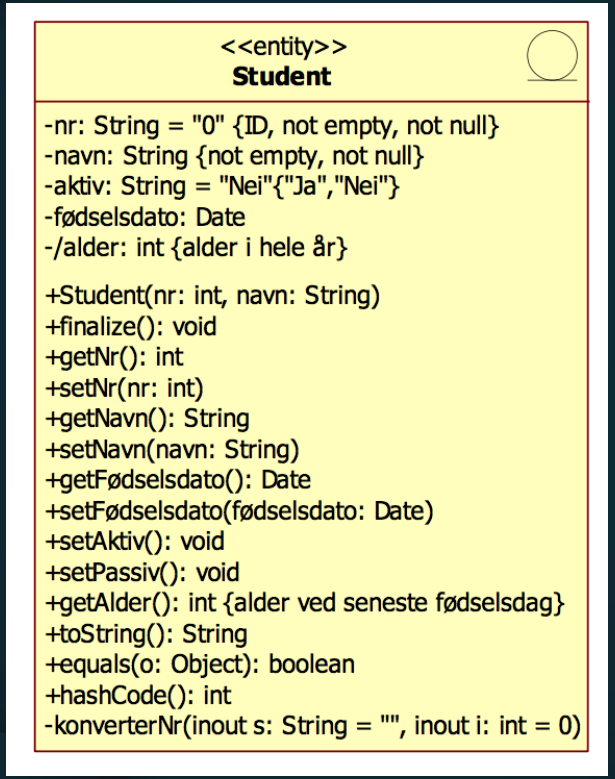
# Design class stereotypes

4. A **data access class** is used to retrieve data from and send data to a database.



# Design class stereotypes

Example

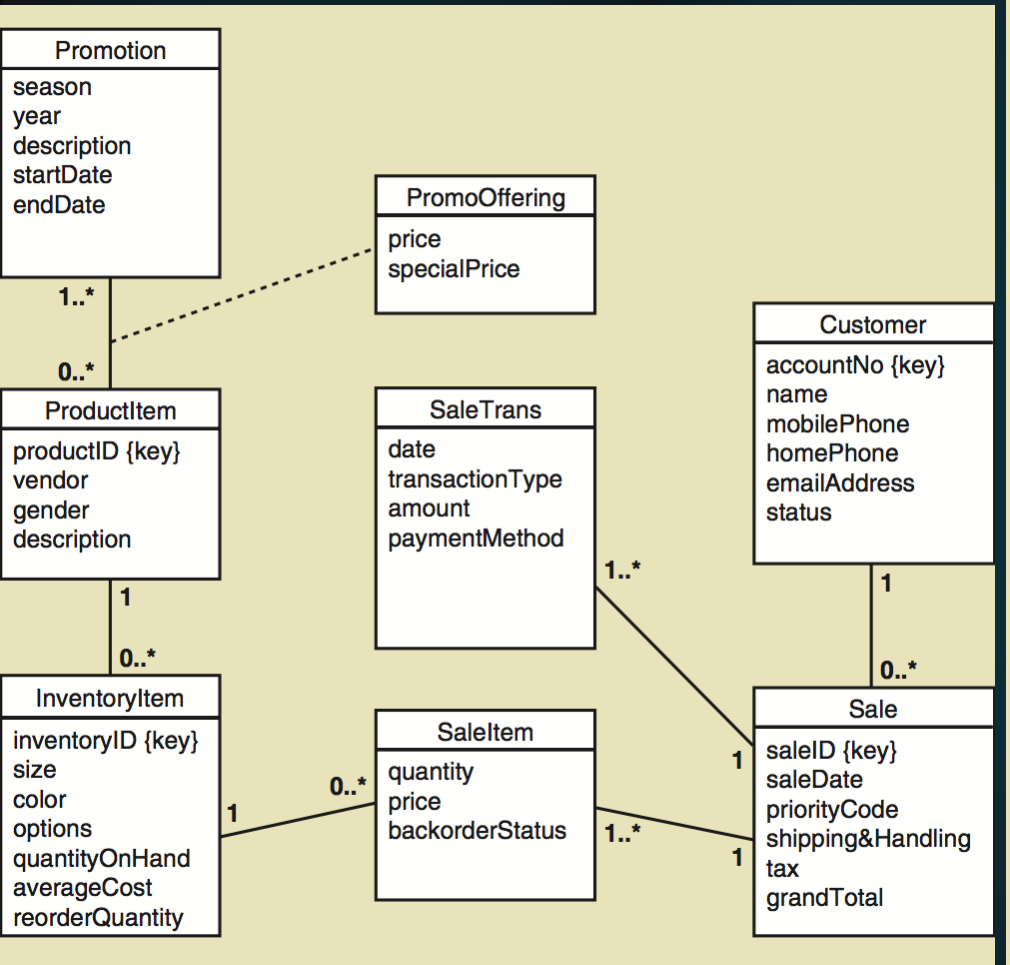


# The first-cut design class diagram

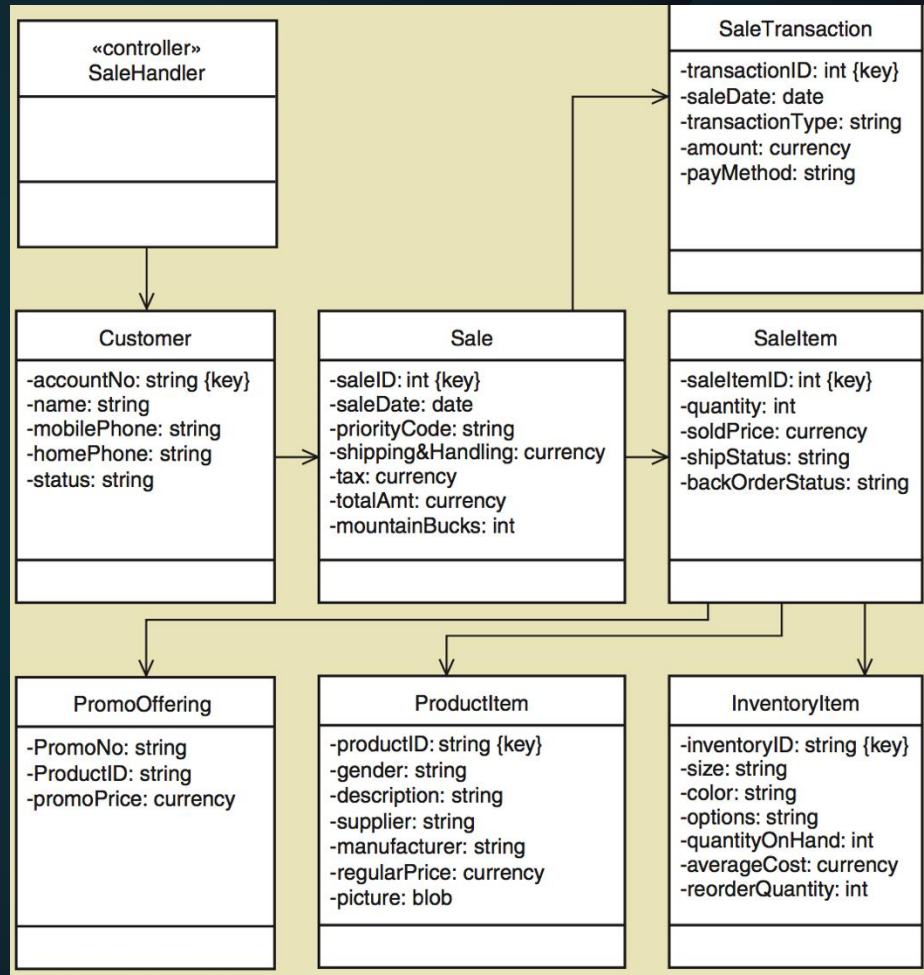
- The first-cut design class diagram is developed by **extending the domain model class** diagram. It requires two steps: (1) adding type and initial value information to the attributes and (2) adding navigation visibility arrows.

- The very first step you do when creating the first-cut design class diagram is to add a controller class and place it between the user-interface classes and the problem domain classes. It has navigation visibility at the top of the visibility hierarchy.

# Domain class diagram



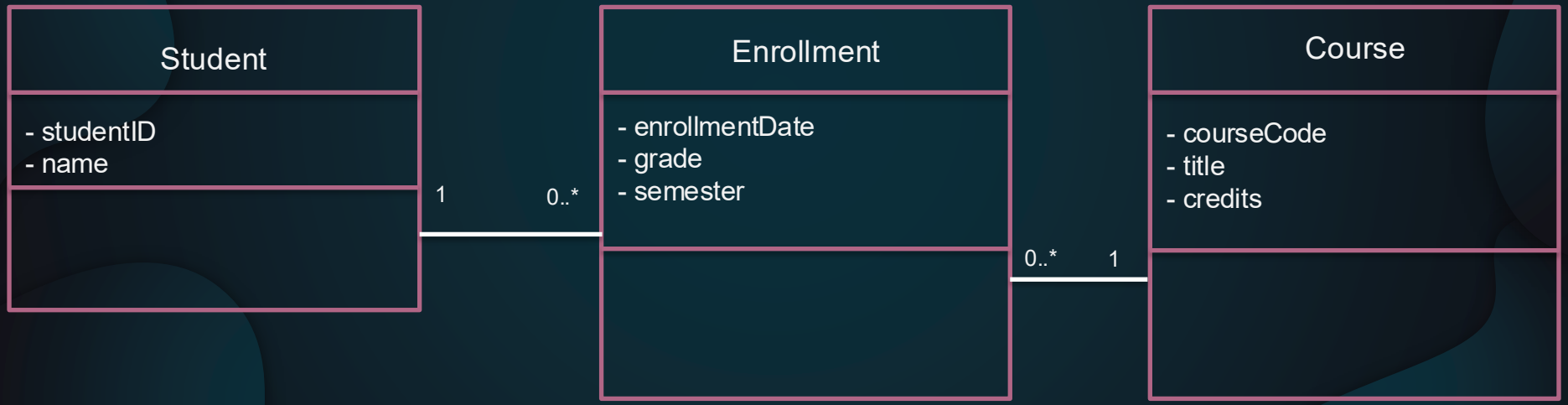

# Design class diagram (first-cut)



In a university enrollment system, a domain model might identify:

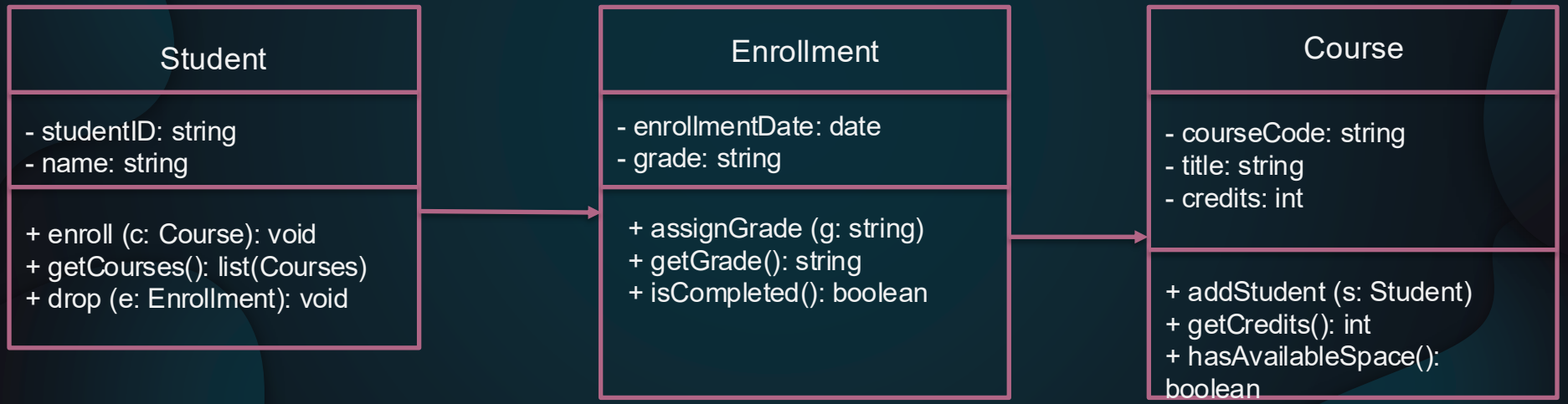
Student —— enrolls in —— Course

What concepts are important in this university domain?



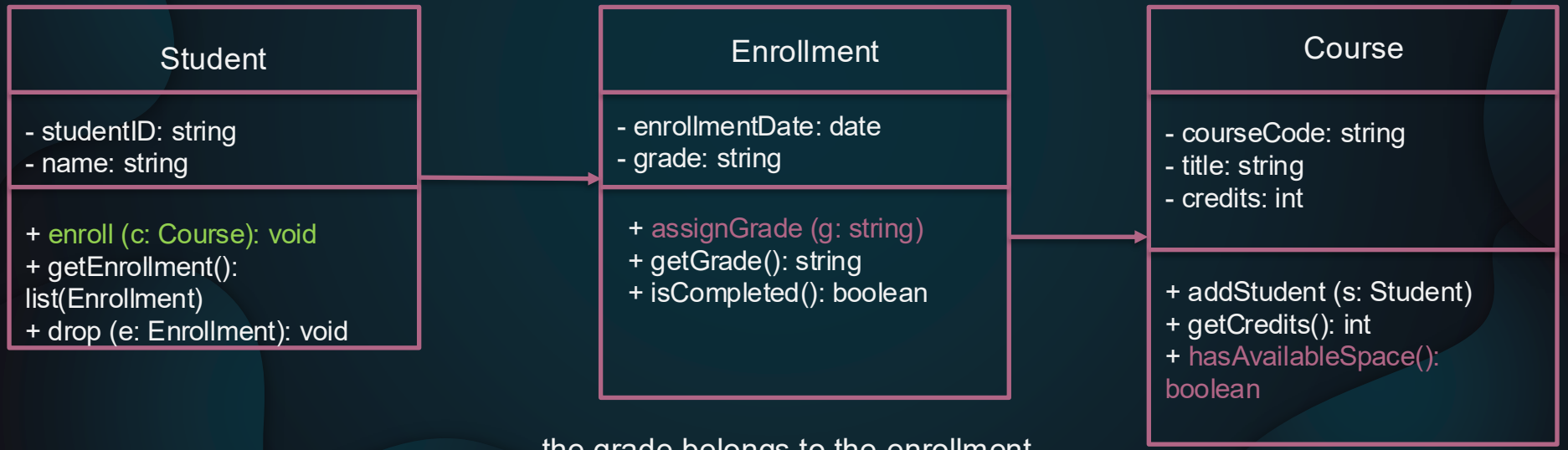

You don't yet need to decide exactly how the programmer will implement them.

You are designing **software behavior**, not describing the domain:



When you assign responsibilities: Which class should be responsible for this behavior?

Give a responsibility to the class that has the information needed to perform it.




the grade belongs to the enrollment

the course knows about its capacity

**"Who owns the information?"**

That's about **attributes**.

Who knows the student's name? – Student

Who knows the course's title? – Course

Who knows the grade? - Enrollment

**"Who has the information needed to make a decision?"**

That's about **responsibility**.

Is this course full?

Course
+ hasAvailableSpace(): boolean

Enroll Student X in Course Y.

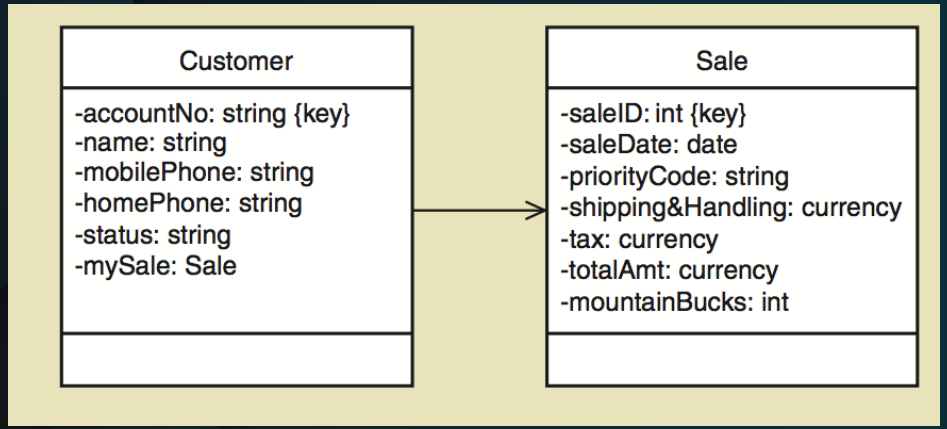
**"Who should coordinate an operation involving several objects?"**

That might be a **service/controller** rather than one of the domain objects.

EnrollmentService
+ enroll(student, course): Enrollment

# Navigation visibility

Which classes need to have references to or be able to access which other classes?




One thing to remember about navigation visibility is that the **classes are programming classes**, not database classes.

Elaboration of attributes: all attributes are kept private

Navigation visibility: is a design principle in which one object has a reference to another object and thus can interact with it.

- one-way navigation visibility: Customer can interact with Sale
- navigation arrow: the Sale object must be visible to the Customer object
- mySale attribute: programming reference to the Sale object.

1. Attribute navigation visibility: occurs when a class has an attribute that references another object. (mySale)

2. Parameter navigation visibility: occurs when a class is passed a parameter that references another object.

- is usually passed through a method call.

# Attribute navigation visibility

A Sale object needs to know which Payment belongs to it.

The Sale object stores a reference to a Payment object as an attribute.




mySale.payment

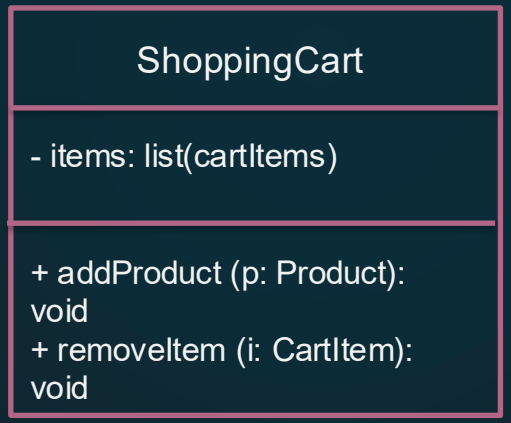
allows mySale to navigate to the associated Payment

I have a Sale object stored inside me.

# Parameter navigation visibility

A ShoppingCart object needs to know which Products belongs to it.

When adding a product cart.addProduct(product) the Product is passed into addProduct().




Give me a Product object when you call this method, and I'll use it.

# Guidelines for navigation visibility

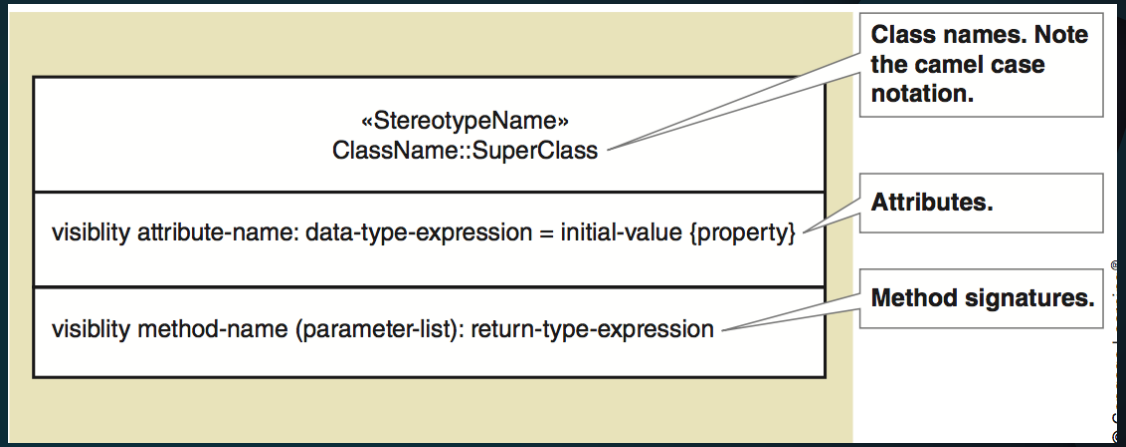
1. One-to-many associations that indicate a superior/subordinate relationship are usually navigated from the superior to the subordinate—for example, from Sale to SaleItem.
2. Mandatory associations, where objects in one class can't exist without objects of another class, are usually navigated from the independent class to the dependent class—for example, from Customer to Sale.
3. When an object needs information from another object, a navigation arrow might be required, pointing either to the object itself or to its parent in a hierarchy.
4. Navigation visibility arrows may be bidirectional—for example, a Sale object might need to send a message to its Customer object as well as the reverse.

# Design class diagram

The name compartment:
stereotype name, class name, and superclass name (if any).

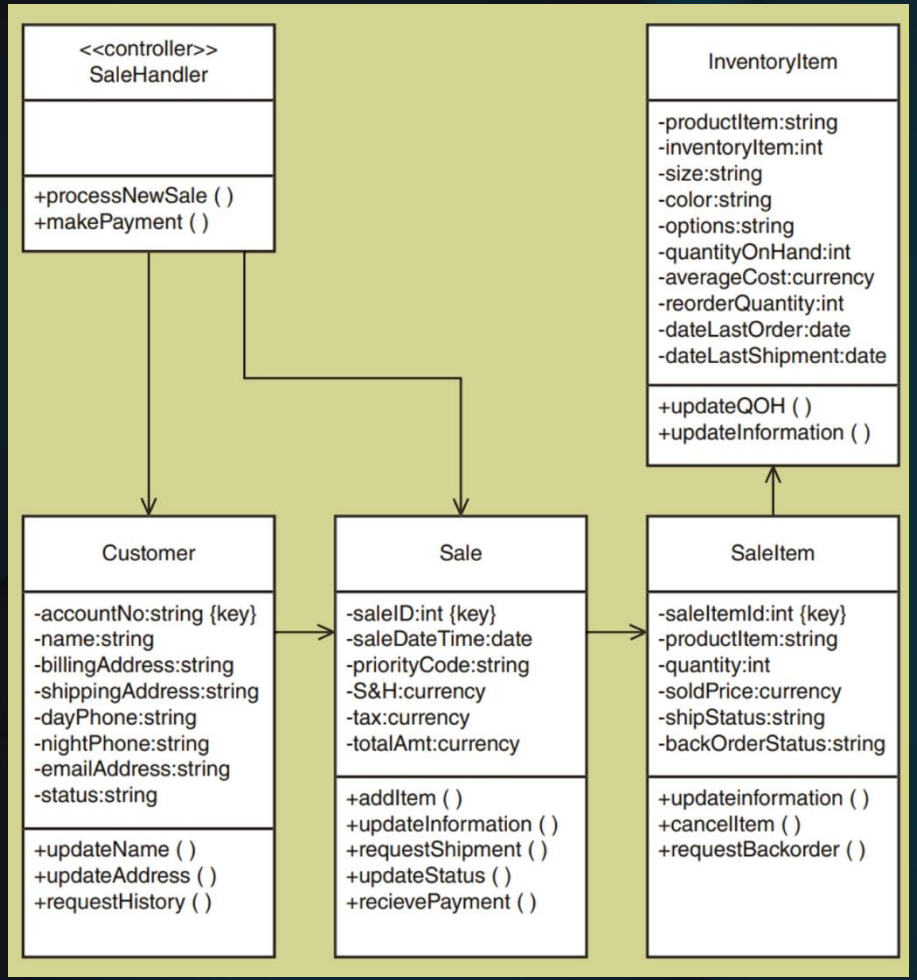
Attributes:
- Attribute name
- Data-type-expression (such as character, string, integer, number, currency, or date)
- Initial-value, if applicable
- Property (within curly braces), such as {key}, if applicable

Method signature:
- Method name
- Method parameter list (incoming arguments)
- Return type-expression (the type of the return parameter from the method)




*[true/false condition] return-value := message-name (parameter-list)*

Message-name is the description of the requested service written as a verb-noun.
Parameter-list shows the data that are passed with the message.
Return-value describe data being returned from the destination object to the source object




# Methods

A **method** is the equivalent of a function in object-oriented programming.

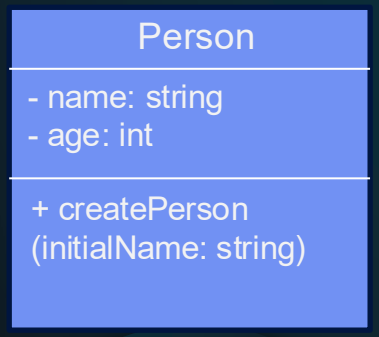
A method accepts parameters as arguments, manipulates these, and then produces an output when the method is called on an **object**.

Methods are classified according to their purpose in the class design:

- interface methods: what other objects can ask the object to do;
- constructor methods: how a new object is created;
- implementation methods: internal work the object does to accomplish something.

```java
public class Person {
    private String name;
    private int age;

    public Person(String initialName) {
        this.name = initialName;
        this.age = 0;
    }
}
```




# Constructor methods

A constructor is used to create and instantiate a new object.

How do I create an object of this class?

- usually named after the class

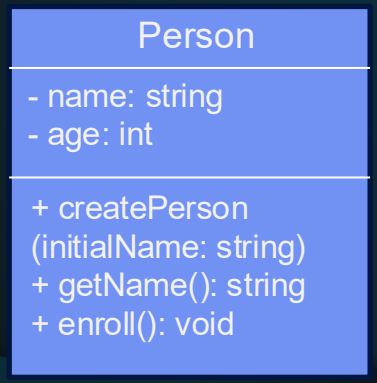
- the parameters of a constructor method, when present, are usually the initial values to be assigned to some or all of the attributes of that object.

- constructors are listed first and then all class methods.

```java
public class Person {
    private String name;
    private int age;

    public Person(String initialName) {
        this.name = initialName;
        this.age = 0;
    }

    public String getName() {
        return this.name;
    }
}
```




# Interface methods

Interface methods are the methods that have the purpose of providing an interface for an object with the external environment.

What can this object do for other objects?

- e.g., **getter** and **setter** methods

- retrieve and update attribute values

- A getter gets the variable from the user input, and the setter assigns the variable as it will be used throughout the class.

- Getters and setters are methods which provide internal attributes access to external agents in an indirect way.

# Implementation methods

```
class Square(object):

    # Constructor:
    def __init__(self):
        self.sideLength = 5.0

    def getSideLength(self):
        return self.sideLength

    def setSideLength(self,sideLength):
        self.sideLength = sideLength

    # Implemented functionality:
    def outputArea():
        return self.getSideLength() * self.getSideLength()

square = Square()

square.setSideLength(3.0)

print square.outputArea() # prints 9.0 to the screen
```

An implementation method is a method used internally to make the class's behavior work.

What does the object need to do internally to accomplish its responsibilities?

The implementation method is the one that most resembles a standard procedural function.

*enroll(course):*

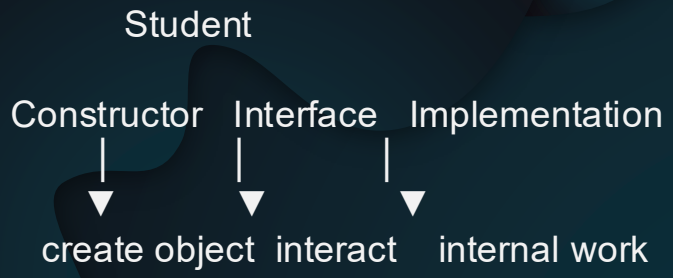
*if checkPrerequisites(course) == false:*

*reject enrollment*

*else:*

*create enrollment*

# All three method groups put together




| Student |
| --- |
| - studentId: string<br>- name: string |
| + createStudent(id: String, name: String)<br>+ enroll(c: Course): void<br>+ getName(): string<br>- checkPrerequisites(c: Course)<br>- createEnrollment(c: Course) |

Step 1 — Create the Student

createStudent("S123", "Anna") – you have created an object.

Step 2 — Ask Student to enroll

anna.enroll(databaseSystems) - another object interacts with Student.

Step 3 — Student does internal work

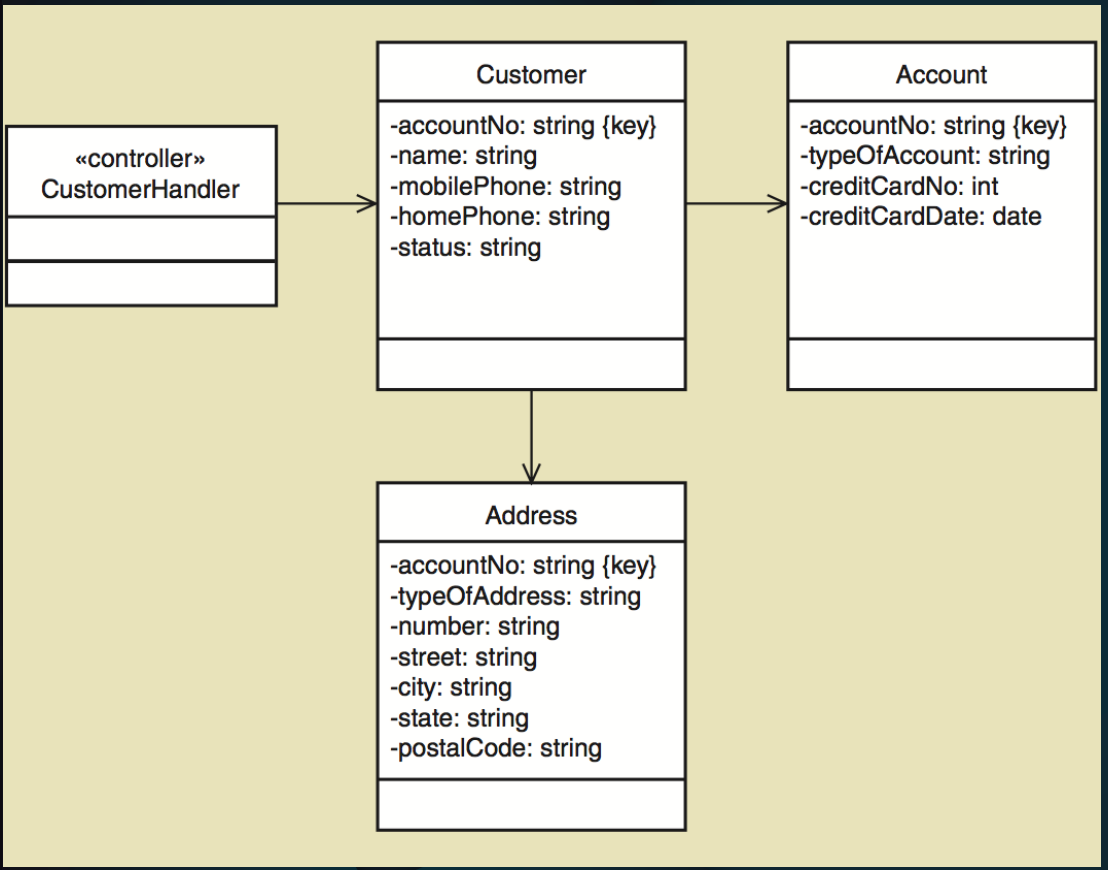
checkPrerequisites(databaseSystems) then

createEnrollment(databaseSystems)

**Remember:** the classification depends on the role the method plays in the design, not simply on the method's name.

| Method | Type | Why? |
| --- | --- | --- |
| Student(id, name) | **Constructor** | Creates/initializes a Student |
| enroll(course) | **Interface** | Other objects can ask Student to enroll |
| getName() | **Interface** | Other objects can request the name |
| checkPrerequisites(course) | **Implementation** | Internal logic supporting enrollment |
| calculateFinalGrade() | Could be **interface or implementation** | Depends on whether other objects need to invoke it |

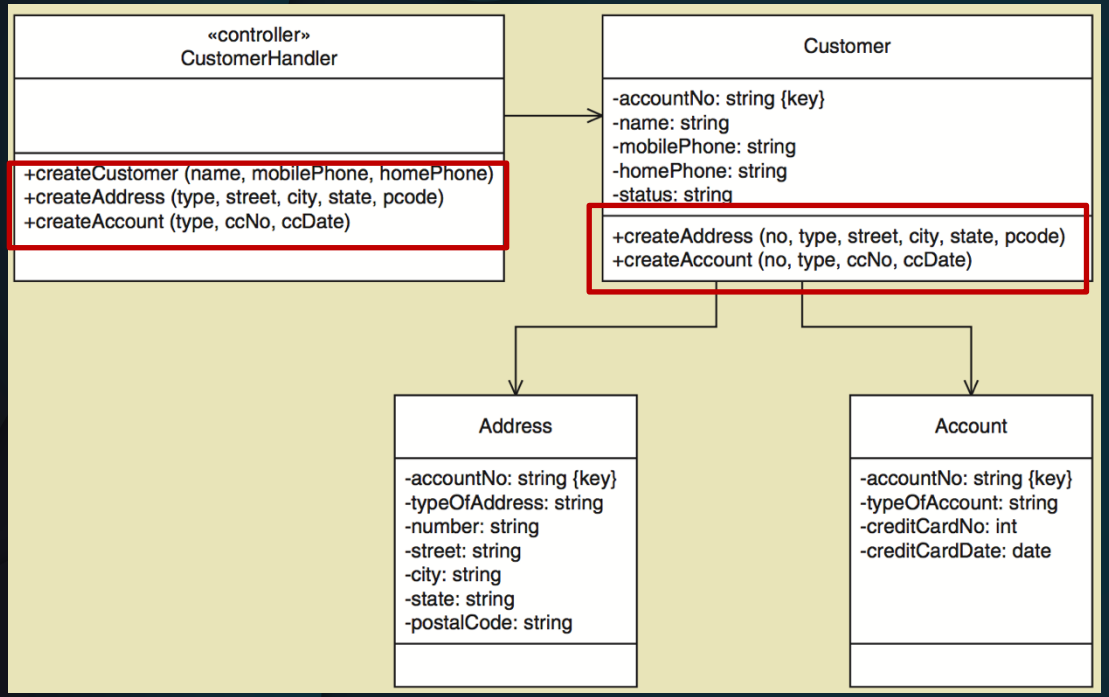
# First-cut design class diagram




1. First, identify the **problem domain class that has responsibility** for this use case. This class will receive the **first message** from the **use case controller**.

2. Second, identify **other classes** that must collaborate with the primary class to complete the use case.

# Updated class diagram




The methods in the design class diagram are derived from the lists of responsibilities on each card. In other words, each **responsibility should translate to a method**.

The CRC cards identify responsibilities, but do not normally include the complete method signatures with **parameters and returns**.

Does anyone have any questions?

# THANK YOU!

katerina.mangaroska@usn.no